

Sistema Ponto de Venda [PDV] — EP1

Isabella De Sousa Santana — 16941052

Lucas Correa Meira — 15695588

9 de outubro de 2026

1 Proposta

O projeto consiste no desenvolvimento de um sistema de Ponto de Venda (PDV) integrado, com suporte a transações em tempo real, controle de estoque, autenticação via OAuth2 e pagamentos integrados (como Mercado Pago).

Com o intuito de ser acessível a qualquer plataforma em que estiver, a proposta usufrui de uma aplicação via web/navegadores usando FullStack puro (HTML, CSS, JavaScript) para o front-end (parte do navegador) e Node.js + SquirrelSQL na questão da manipulação dos tráfegos e informações de bancos de dados (incluso serviços externos ao escopo atual, mas requeridas pelo desenvolvimento como um todo).

Atendendo aos requisitos do EP1, este projeto reconstituiu a implementação de todo servidor HTTP e WebSocket operando diretamente sobre sockets TCP puros no Node.js, dispensando bibliotecas de alto nível como Express ou Socket.io. O sistema é containerizado via Docker.

2 Decisões de Implementação e Justificativa

Primeiramente, fez o uso de uma arquitetura Cliente-Servidor (CS) estrita. O front-end (cliente) faz requisições HTTP RESTful para operações assíncronas (compras, listagens) e mantém uma conexão WebSocket persistente para receber atualizações de estado (ex: pagamentos recebidos, atualização de comandas).

Nela, só há aplicação pura do Protocolo de Transporte (TCP); como o sistema lida com transações financeiras, persistência no banco de dados SQLite e autenticação via tokens (OAuth2), a confiabilidade do TCP (garantia de entrega e ordenação dos pacotes) é mandatória. O UDP não é adequado para este cenário, pois a perda de pacotes poderia resultar em falhas de transação ou dados corrompidos de clientes.

Seguindo o reiterado na propostas, implementou-se um micro-framework em socket.js, no qual interpreta pacotes TCP brutos, realiza o parsing de cabeçalhos HTTP e gerencia o upgrade de conexão para WebSocket.

3 Protocolo

O sistema implementa o protocolo HTTP/1.1 e o protocolo WebSocket (RFC 6455) de forma nativa. No código, pode-se mencionar os seguintes componentes:

3.1 Sintaxe:

As mensagens de aplicação (payloads) trafegam estritamente no formato JSON. No nível de WebSocket, os frames obedecem à sintaxe binária padrão, onde o primeiro byte define o FIN e o Opcode (ex: 0x81 para texto final), e os bytes subsequentes definem o tamanho do payload e a máscara de decodificação (client-to-server).

3.2 Semântica:

Rotas HTTP: Ações bem definidas como `/getProducts`, `/addPurchase`, `/pay-Maquinhinha`, onde os status codes (200, 400, 401, 500) indicam o sucesso ou falha da operação.

3.3 Eventos WS:

Sinais semânticos de notificação, como "update!" ou "payment-received!", indicando aos clientes que devem buscar novamente o estado do servidor.

3.4 Gramática:

O parser implementado em `socket.js` exige a gramática clássica HTTP: uma linha de requisição (METHOD URL PROTOCOL), seguida por cabeçalhos separados por `\r\n`, finalizada por uma linha em branco (`\r\n\r\n`) antes do body.

4 Rotas HTTP API

Todas as rotas HTTP implementadas utilizam o método POST e dependem do middleware `jsonParser` para interpretar o corpo da requisição de forma segura.

POST `/getRevenue`
Segurança: Rota pública.
Comportamento: Acessa a base SQLite e executa uma query de soma total agregada (`SELECT COALESCE(SUM(total), 0)`).
Retorno: JSON com o valor financeiro total consolidado de todas as vendas (ex: `{ "total": 1500.50 }`).

POST /getReviews

Segurança: Rota pública.

Comportamento: Realiza um JOIN relacional no SQLite entre as tabelas reviews e sales, ordenando cronologicamente.

Retorno: Array de objetos com dados de avaliação: saleExternalId, reviewer, stars (1-5) e comment.

POST /getSales

Segurança: Pública, porém sensível.

Comportamento: Busca as comandas (sales) e os itens (sale_items). O motor JavaScript associa os itens às suas respectivas comandas, faz o parse da coluna JSON extras para objetos estruturados e converte flags booleanas (como delivered).

Retorno: Array abrangente contendo o espelho dos caixas (comprador, mesa, total financeiro, status de entrega e itens).

POST /oauth/authenticate

Segurança: Endpoint principal de autenticação (Pública).

Comportamento: Usa a engine OAuth2 para validar credenciais cruzando com tokens.json. Em caso de sucesso, emite um Opaque Token.

Retorno (Stateful): Injeta o cabeçalho Set-Cookie: session_token=...; HttpOnly; Secure; SameSite=strict. Isso estabelece a sessão persistente para as rotas protegidas subsequentes.

POST /addPurchase

Segurança: Protegida (requireAuth).

Comportamento (Core): Processa transações de venda. Recebe buyer, comment e products. Calcula o valor financeiro do lado do servidor (protegendo contra fraudes de preço no client), invoca takeStock para deduzir o inventário, gera externalId (UUID) e um número de comanda sequencial (tabNumber).

Side-Effect: Ao finalizar a transação no SQLite com sucesso, aciona update_ws() para disparar um broadcast para todos os terminais.

Retorno: { ok: true, externalId, tabNumber, total }. Retorna HTTP 409 em caso de estoque insuficiente.

POST /deliverSale

Segurança: Protegida.

Comportamento: Recebe um número de comanda, limpa caracteres não-numéricos (usando regex \D) e faz um UPDATE na base setando delivered = 1.

Side-Effect: Dispara broadcast WS update! para remover a comanda da tela da cozinha.

POST /addReview

Segurança: Protegida.

Comportamento: Valida o input de estrelas (1 a 5). Busca a venda correspondente pelo external_id e persiste a avaliação na tabela reviews conectada àquela venda.

POST /payMaquininha

Segurança: Protegida.

Comportamento: Integração direta com a API Mercado Pago Point. Recalcula os preços localmente no DB, monta o payload no formato MP (PIX, Crédito, Débito) e dispara uma chamada externa (usando UUID para idempotência) enviando o comando para o terminal físico.

POST /payment_WebHook

Segurança: Protegida.

Comportamento: Armazena na tabela payment_webhooks um mapeamento relacional entre o identificador da intenção de compra externa e o hooker_id (sessão WebSocket da tela específica que gerou o QR Code).

POST /callbackML

Segurança: Validação HMAC SHA-256 via header x-signature.

Comportamento: Endpoint exposto ao túnel Cloudflared. Confirma aprovação de pagamentos consultando a API do Mercado Pago, cruza com o mapeamento salvo no banco e envia uma notificação WebSocket direcionada especificamente para o cliente de origem, ordenando a impressão do recibo.

5 Protocolo WebSocket

WS Handshake Inicial

Gatilho: Cliente envia requisição com header Upgrade: websocket.

Mensagem: { "data": "added_client", "UNIQUE_ID": "..." }

Semântica: O servidor registra a aba do navegador no vetor CLIENTS com um UUID único efêmero, que é salvo pelo frontend para receber callbacks direcionados de pagamento.

WS Broadcast (update!)

Gatilho: Invocado após mudanças de estado globais (addPurchase, deliverSale).

Mensagem: "update!" (Texto puro)

Semântica: O servidor faz varredura em todos os sockets ativos e envia a string. O frontend a interpreta como um sinal para invalidar o cache visual e disparar uma requisição HTTP de fundo (/getSales) atualizando a tela sem refresh físico.

WS Unicast (payment-received!)

Gatilho: Confirmação validada recebida na rota /callbackML.

Mensagem: { "data": "payment-received!", "name": "..." }

Semântica: Roteada exclusivamente para o socket que originou o pagamento. Destrava o loading na interface de autoatendimento e inicializa o fluxo final de aprovação visual para o comprador.

6 Como Ler o Código

Para entender o fluxo do projeto, recomenda-se a seguinte ordem de leitura:

1. docker-compose.yml: Ponto de partida da infraestrutura. Define os containers do frontend (Apache), backend (Node.js), banco de dados, e túnel reverso (Cloudflared);
2. socket.js e websocket.js: O coração da infraestrutura de rede. Veja como a classe MiniExpress sobe um servidor TCP (net.createServer), bufferiza os pacotes brutos, separa os cabeçalhos HTTP e faz o tratamento da chave Sec-WebSocket-Key para aceitar conexões bidirecionais;
3. Back-End.js (Main): O entrypoint da aplicação. Aqui o framework MiniExpress é instanciado. O arquivo contém a configuração do banco SQLite, a lógica de OAuth2, as rotas de venda (ex: createSale usando transações), e o Webhook do Mercado Pago;

4. Middle.js: Camada de serviço no Frontend. Demonstra como o cliente consome a API do backend através do wrapper `accessBACK` e lida com o parse de dados (como os objetos `DADOS_PRODUTOS` e controle de comandas).

OBS.: MAIS INSTRUÇÕES USE O README.MD

7 Como Usar a Aplicação

1. Garanta que o Docker e o Docker Compose estejam instalados na máquina;
2. Na raiz do projeto, execute o comando: `docker-compose up -d`;
3. O frontend estará acessível via `http://localhost:85` (ou, caso o server exterior esteja aberto, em `usp.perimin.com.br`);
5. Para integrações externas (como o Webhook de pagamentos), o container do cloudflared estabelecerá automaticamente um túnel, permitindo que a API local receba callbacks do Mercado Pago.

8 Testes e Resultados.

Os testes foram modularizados em dois âmbitos (observáveis pelas flags no código):

- Modo de Teste (Mock): A flag `TEST_MODE` em Middle.js e as verificações `content.test_mode` no backend permitem injetar preços fictícios (ex: 0.01) para testar a comunicação completa com a API de pagamentos sem custos reais;
- Testes de Rede (Concorrência): O buffer de TCP implementado lida com a fragmentação de pacotes de forma robusta, esperando até que o `\r\n\r\n` seja recebido para fatiar o body;
- Resultados Finais: O sistema consegue gerenciar múltiplas sessões WebSocket simultâneas. Notificações de pagamento chegam via túnel Cloudflared, são processadas validando a assinatura HMAC (x-signature), o banco SQLite é atualizado e as comandas na interface cliente sofrem broadcast instantâneo.

As diversas condições de testes possíveis foram testadas; pode-se mencionar algumas delas (mas não se limitando a somente a isso):

- Requisição HTTP Simples (GET/POST):
 - Ação: Fazer um POST válido (ex: `‘/getRevenue‘` via Postman ou Frontend).

- Resultado Esperado: O ‘socket.js’ identifica os headers (‘\r\n\r\n’), extrai o body e retorna JSON 200 OK.
- Fragmentação de Pacotes TCP (Stress Test):
 - Ação: Enviar um payload JSON muito grande (ex: muitos produtos na compra) usando uma rede lenta para forçar pacotes TCP divididos.
 - Resultado Esperado: O ‘socket.js’ deve acumular o ‘buffer’ (linha ‘buffer.length < totalRequiredLength’) sem quebrar, processando apenas quando receber 100% do Content-Length.
- Rotas Inexistentes:
 - Ação: Fazer um GET para ‘/rota-aleatoria’.
 - Resultado Esperado: Retorno HTTP 404 "Route not found".
- Login Incorreto:
 - Ação: Tentar ‘/oauth/authenticate’ com credenciais falsas.
 - Resultado Esperado: Rejeição da requisição (HTTP 4xx).
- Login Correto (Geração de Cookie):
 - Ação: Fazer login válido.
 - Resultado Esperado: Servidor retorna header ‘Set-Cookie: session_token=...; HttpOnly; Secure; SameSite=strict’.
- Proteção de Rotas (Middleware ‘requireAuth’):
 - Ação: Chamar ‘/getAccountInfos’ SEM o cookie na requisição.
 - Resultado Esperado: HTTP 401 "No token provided".
- Acesso Autenticado:
 - Ação: Chamar ‘/getAccountInfos’ COM o cookie.
 - Resultado Esperado: HTTP 200 OK com os dados do usuário.
- Adição de Compra com Estoque:
 - Ação: Chamar ‘/addPurchase’ solicitando 1 "geladinho baunilha" (que possui 32 no estoque, conforme mock).
 - Resultado Esperado: O SQLite debita 1 do estoque (‘takeStock.run’). Retorna 200 OK com a nova Comanda.
- Entrega de Comanda:
 - Ação: Chamar ‘/deliverSale’ com o ID da comanda.

- Resultado Esperado: O DB marca ‘delivered = 1’ e dispara a notificação via WS.
- Handshake e Upgrade:
 - Ação: O cliente (browser) tenta conectar no ‘ws://localhost:3005/ws’.
 - Resultado Esperado: O ‘socket.js’ calcula a ‘Sec-WebSocket-Accept’ certa e retorna HTTP 101 Switching Protocols.
- Registro de Clientes:
 - Ação: Abrir 3 abas do navegador no PDV.
 - Resultado Esperado: O array ‘CLIENTS’ no ‘Back-End.js’ deve ter 3 objetos com ‘UNIQUE_ID’ gerados pelo crypto.
- Broadcast de Atualização:
 - Ação: Completar uma compra ou entregar uma comanda em uma aba.
 - Resultado Esperado: O método ‘update_ws()’ faz um ‘send("update!")’ para todas as outras abas, e o frontend atualiza a tela sem refresh manual.
- Desconexão Limpa:
 - Ação: Fechar a aba do navegador.
 - Resultado Esperado: O frame binário ‘0x08’ (Close) é lido no ‘parse-Frame’, a conexão fecha e o objeto é removido do array ‘CLIENTS’.

9 Fontes

- RFC 6455 - The WebSocket Protocol (IETF);
- RFC 7230 - HTTP/1.1 Message Syntax and Routing;
- Documentação da API Node.js (Módulos net, crypto, events);
- Documentação Oficial do SQLite3 (Comportamento transacional e locks);
- Referência de Integração e Webhooks do Mercado Pago (In-person payments).